

MongoDB con C++

Guía Completa y Detallada

mongocxx • Conexión • CRUD • Filtros • Documentos BSON • Aggregation

1. Instalación del Driver mongocxx

El driver oficial de MongoDB para C++ se llama mongocxx. Depende de bsoncxx para manejar documentos BSON.

1.1 Instalar en Ubuntu/Debian

```
# Instalar dependencias
sudo apt-get install libmongoc-1.0-0 libbson-1.0
sudo apt-get install libssl-dev libsasl2-dev cmake

# Instalar mongocxx desde el repositorio
sudo apt-get install libmongocxx-dev

# O compilar desde fuente (versión más reciente):
wget https://github.com/mongodb/mongo-cxx-driver/releases/download/r3.10.1/mongo-cxx-driver-r3.10.1.tar.gz
tar -xzf mongo-cxx-driver-r3.10.1.tar.gz
cd mongo-cxx-driver-r3.10.1/build
cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/usr/local
sudo make install
```

1.2 Compilar tu programa con el driver

```
# Compilar con pkg-config (recomendado)
g++ main.cpp -o programa \
    $(pkg-config --cflags --libs libmongocxx) \
    $(pkg-config --cflags --libs libbsoncxx)

# O especificando las rutas manualmente
g++ main.cpp -o programa \
    -I/usr/local/include/mongocxx/v_noabi \
    -I/usr/local/include/bsoncxx/v_noabi \
    -lmongocxx -lbsoncxx
```

1.3 Encabezados necesarios

```
#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <mongocxx/uri.hpp>
#include <mongocxx/database.hpp>
#include <mongocxx/collection.hpp>
#include <bsoncxx/builder/stream/document.hpp>
#include <bsoncxx/builder/basic/document.hpp>
#include <bsoncxx/json.hpp>

using namespace std;
using bsoncxx::builder::stream::document;
using bsoncxx::builder::stream::finalize;
using bsoncxx::builder::stream::open_document;
using bsoncxx::builder::stream::close_document;
using bsoncxx::builder::stream::open_array;
using bsoncxx::builder::stream::close_array;
```

💡 *mongocxx::instance DEBE crearse una sola vez antes de cualquier operación. Se crea una vez en el main().*

2. Conexión a MongoDB

2.1 Conexión básica local

```
#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <mongocxx/uri.hpp>

int main() {
    // OBLIGATORIO: crear instancia una sola vez
    mongocxx::instance inst{};

    // Conectar a MongoDB local
    mongocxx::client client{mongocxx::uri{"mongodb://localhost:27017"}};

    // Seleccionar base de datos (se crea al insertar el primer documento)
    mongocxx::database db = client["silkroad"];

    cout << "Conectado a MongoDB" << endl;

    return 0;
    // El cliente se cierra automáticamente al salir del scope
}
```

2.2 Conexión con usuario y contraseña

```
mongocxx::instance inst{};

string uri_str = "mongodb://usuario:contraseña@localhost:27017/silkroad?
authSource=admin";
mongocxx::client client{mongocxx::uri{uri_str}};
mongocxx::database db = client["silkroad"];
```

2.3 Conexión con opciones avanzadas (timeout, pool)

```
#include <mongocxx/options/client.hpp>
#include <mongocxx/options/pool.hpp>

mongocxx::instance inst{};

mongocxx::uri uri{"mongodb://localhost:27017"};

mongocxx::options::client client_opts;
// No hay pool en el driver básico, pero sí hay mongocxx::pool

// Para usar pool de conexiones:
mongocxx::pool pool{uri};

// Obtener una conexión del pool
auto entry = pool.acquire();
mongocxx::database db = (*entry)["silkroad"];
```

2.4 Clase de conexión Singleton (patrón MVC)

```
// Conexion.hpp
#pragma once
#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <mongocxx/database.hpp>

class Conexion {
private:
    static mongocxx::instance* inst;
    static mongocxx::client* client;

public:
    static mongocxx::database getDatabase() {
        if (!inst) {
            inst = new mongocxx::instance{};
            client = new
mongocxx::client{mongocxx::uri{"mongodb://localhost:27017"}};
        }
        return (*client)["silkroad"];
    }
};

// Conexion.cpp
mongocxx::instance* Conexion::inst = nullptr;
mongocxx::client* Conexion::client = nullptr;
```

3. Crear Documentos BSON

En C++ los documentos MongoDB se crean con el builder de bsoncxx. Hay dos estilos:

3.1 Stream builder (encadenado con <<)

```
#include <bsoncxx/builder/stream/document.hpp>
using bsoncxx::builder::stream::document;
using bsoncxx::builder::stream::finalize;

// Crear documento
auto doc = document{}
    << "nombre" << "Carlos"
    << "cedula" << 1234567
    << "correo" << "carlos@email.com"
    << "precio" << 29.99
    << "activo" << true
    << finalize; // <-- OBLIGATORIO: cierra y construye el documento

// doc es de tipo bsoncxx::document::value
```

3.2 Basic builder (make_document)

```
#include <bsoncxx/builder/basic/document.hpp>
#include <bsoncxx/builder/basic/kvp.hpp>
using bsoncxx::builder::basic::make_document;
using bsoncxx::builder::basic::kvp;

// Forma más moderna y segura
auto doc = make_document(
    kvp("nombre", "Carlos"),
    kvp("cedula", 1234567),
    kvp("correo", "carlos@email.com"),
    kvp("precio", 29.99),
    kvp("activo", true)
);
```

3.3 Documento con objeto anidado

```
using bsoncxx::builder::stream::open_document;
using bsoncxx::builder::stream::close_document;

auto doc = document{}
    << "nombre" << "Carlos"
    << "direccion" << open_document
        << "calle" << "Av. Principal 123"
        << "ciudad" << "Quito"
        << "pais" << "Ecuador"
    << close_document
    << finalize;
```

3.4 Documento con array

```
using bsoncxx::builder::stream::open_array;
using bsoncxx::builder::stream::close_array;

auto doc = document{}
    << "nombre" << "Carlos"
    << "tags" << open_array
        << "premium" << "activo" << "cliente"
    << close_array
    << finalize;
```

Tipo C++	Tipo BSON	Ejemplo en builder
string / const char*	UTF-8 String	<< "campo" << "texto"
int32_t / int	Int32	<< "campo" << 42
int64_t / long	Int64	<< "campo" << (int64_t)123456789
double	Double	<< "campo" << 3.14
bool	Boolean	<< "campo" << true
bsoncxx::oid	ObjectId	<< "_id" << bsoncxx::oid{}
bsoncxx::types::b_null	null	<< "campo" << bsoncxx::types::b_null{}
open_document / close_document	Embedded Document	Ver ejemplo 3.3
open_array / close_array	Array	Ver ejemplo 3.4

4. Operaciones CRUD

4.1 INSERT — Insertar documentos

insert_one() — Insertar un documento

Retorna: `mongocxx::result::insert_one` (wrapped en `stdx::optional`)

```
mongocxx::database db = Conexion::getDatabase();
mongocxx::collection col = db["usuarios"];

auto doc = make_document(
    kvp("nombre", "Carlos"),
    kvp("correo", "carlos@email.com"),
    kvp("cedula", 1234567)
);

auto resultado = col.insert_one(doc.view());

// Lo que retorna (stdx::optional<mongocxx::result::insert_one>):
if (resultado) {
    // .inserted_id() → bsoncxx::types::bson_value::view — el _id generado
    auto id = resultado->inserted_id();
    cout << "ID: " << id.get_oid().value.to_string() << endl;
}
```

insert_many() — Insertar varios documentos

Retorna: `stdx::optional<mongocxx::result::insert_many>`

```
#include <vector>

vector<bsoncxx::document::value> docs;
docs.push_back(make_document(kvp("nombre", "Ana"),    kvp("precio", 29.99)));
docs.push_back(make_document(kvp("nombre", "Luis"),   kvp("precio", 49.99)));
docs.push_back(make_document(kvp("nombre", "Maria"),  kvp("precio", 19.99)));

auto resultado = col.insert_many(docs);

// Lo que retorna:
if (resultado) {
    // .inserted_count() → int64_t — cuántos documentos se insertaron
    cout << "Insertados: " << resultado->inserted_count() << endl;

    // .inserted_ids() → map<size_t, bsoncxx::types::bson_value::view>
    //                  mapa posición → _id de cada documento
}
```

4.2 FIND — Buscar documentos

find() — Buscar múltiples documentos

Retorna: `mongocxx::cursor` (iterador sobre los documentos)

```
#include <bsoncxx/json.hpp> // para to_json()

// Buscar todos los documentos
```



```

mongocxx::cursor cursor = col.find({});

// Iterar con range-based for
for (const auto& doc : cursor) {
    // doc es bsoncxx::document::view
    cout << bsoncxx::to_json(doc) << endl;
}

// Buscar con filtro
auto filtro = make_document(kvp("rol", "CLIENTE"));
mongocxx::cursor resultado = col.find(filtro.view());

for (const auto& doc : resultado) {
    string nombre = string(doc["nombre"].get_string().value);
    cout << nombre << endl;
}

```

find_one() — Obtener solo el primer resultado

Retorna: `stdx::optional<bsoncxx::document::value>`

```

auto filtro = make_document(kvp("correo", "carlos@email.com"));

auto resultado = col.find_one(filtro.view());

// SIEMPRE verificar si encontró algo
if (resultado) {
    // resultado es stdx::optional – usar -> para acceder
    auto doc = resultado->view();

    string nombre = string(doc["nombre"].get_string().value);
    int32_t cedula = doc["cedula"].get_int32().value;
    double precio = doc["precio"].get_double().value;
    bool activo = doc["activo"].get_bool().value;

    cout << nombre << " - " << cedula << endl;
} else {
    cout << "No encontrado" << endl;
}

```

Opciones de find() — limit, skip, sort

```

#include <mongocxx/options/find.hpp>

mongocxx::options::find opts;

// Limitar resultados
opts.limit(5);

// Saltar documentos (paginación)
opts.skip(10);

// Ordenar: 1 = ascendente, -1 = descendente
opts.sort(make_document(kvp("precio", -1)).view()); // más caro primero

// Proyección: solo traer ciertos campos
opts.projection(make_document(kvp("nombre", 1), kvp("precio", 1), kvp("_id", 0)).view());

mongocxx::cursor cursor = col.find({}, opts);

```

4.3 Extraer datos de un documento

Método	Tipo C++	Uso
doc["campo"].get_string().value	std::string_view	Leer String — convertir con string()
doc["campo"].get_int32().value	int32_t	Leer entero int
doc["campo"].get_int64().value	int64_t	Leer entero long
doc["campo"].get_double().value	double	Leer decimal
doc["campo"].get_bool().value	bool	Leer booleano
doc["campo"].get_oid().value	bsoncxx::oid	Leer ObjectId
doc["campo"].get_document().value	bsoncxx::document::view	Leer doc anidado
doc["campo"].get_array().value	bsoncxx::array::view	Leer array
bsoncxx::to_json(doc)	std::string	Convertir doc completo a JSON string

```
// Ejemplo completo de extracción
auto doc = resultado->view();

string nombre = string(doc["nombre"].get_string().value);
int32_t cedula = doc["cedula"].get_int32().value;
double saldo = doc["saldo"].get_double().value;
bool activo = doc["activo"].get_bool().value;

// Documento anidado
auto direccion = doc["direccion"].get_document().value;
string ciudad = string(direccion["ciudad"].get_string().value);

// Array
auto tags = doc["tags"].get_array().value;
for (const auto& tag : tags) {
    cout << string(tag.get_string().value) << endl;
}
```

4.4 UPDATE — Actualizar documentos

update_one() — Actualizar el primero que coincide

Retorna: `stdx::optional<mongocxx::result::update>`

```
#include <mongocxx/result/update.hpp>

auto filtro      = make_document(kvp("correo", "carlos@email.com"));
auto actualizacion = make_document(
    kvp("$set", make_document(
        kvp("nombre", "Carlos García"),
        kvp("telefono", "0991234567")
    ))
);

auto resultado = col.update_one(filtro.view(), actualizacion.view());

// Lo que retorna:
if (resultado) {
    // .matched_count()    → int64_t – cuántos coincidieron con el filtro
    // .modified_count()   → int64_t – cuántos fueron realmente modificados
    // .upserted_id()      → optional<bsoncxx::types::bson_value::view>
    cout << "Modificados: " << resultado->modified_count() << endl;
}
```

Operadores de actualización

Operador	Qué hace	Ejemplo en C++
\$set	Asigna valor a un campo	kvp("\$set", make_document(kvp("campo", valor)))
\$unset	Elimina un campo	kvp("\$unset", make_document(kvp("campo", "")))
\$inc	Incrementa un número	kvp("\$inc", make_document(kvp("stock", -1)))
\$mul	Multiplica un número	kvp("\$mul", make_document(kvp("precio", 1.1)))
\$push	Agrega elemento a array	kvp("\$push", make_document(kvp("tags", "nuevo")))
\$pull	Elimina elemento de array	kvp("\$pull", make_document(kvp("tags", "viejo")))
\$addToSet	Agrega sin duplicar	kvp("\$addToSet", make_document(kvp("tags", "x")))

update_many() — Actualizar todos los que coincidan

Retorna: `stdx::optional<mongocxx::result::update>` (mismos métodos)

```
// Poner activo=false a todos los clientes
auto filtro = make_document(kvp("rol", "CLIENTE"));
auto update = make_document(
    kvp("$set", make_document(kvp("activo", false)))
);
```

```
);  
auto r = col.update_many(filtro.view(), update.view());  
cout << "Actualizados: " << r->modified_count() << endl;
```

replace_one() — Reemplazar documento completo

Retorna: stdx::optional<mongocxx::result::replace_one>

```
auto filtro = make_document(kvp("correo", "carlos@email.com"));  
auto nuevo = make_document(  
    kvp("nombre", "Carlos Nuevo"),  
    kvp("correo", "carlos@email.com"),  
    kvp("rol", "ADMIN")  
);  
col.replace_one(filtro.view(), nuevo.view());
```

4.5 DELETE — Eliminar documentos

delete_one() — Eliminar el primer documento que coincida

Retorna: `stdx::optional<mongocxx::result::delete_result>`

```
auto filtro = make_document(kvp("correo", "spam@email.com"));

auto resultado = col.delete_one(filtro.view());

// Lo que retorna:
if (resultado) {
    // .deleted_count() → int64_t – cuántos documentos se eliminaron (0 o 1)
    cout << "Eliminados: " << resultado->deleted_count() << endl;
}
```

delete_many() — Eliminar todos los que coincidan

Retorna: `stdx::optional<mongocxx::result::delete_result>`

```
// Eliminar todos los productos sin stock
auto filtro = make_document(kvp("stock", make_document(kvp("$lte", 0))));
auto r = col.delete_many(filtro.view());
cout << "Eliminados: " << r->deleted_count() << endl;
```

5. Filtros — Tabla de referencia

En C++ los filtros son documentos BSON con operadores de MongoDB:

Operador MongoDB	Equivalente SQL	Ejemplo en C++
{ campo: valor }	campo = valor	make_document(kvp("rol", "ADMIN"))
{ \$ne: valor }	campo != valor	make_document(kvp("estado", make_document(kvp("\$ne", "inactivo"))))
{ \$gt: valor }	campo > valor	make_document(kvp("precio", make_document(kvp("\$gt", 50.0))))
{ \$gte: valor }	campo >= valor	make_document(kvp("edad", make_document(kvp("\$gte", 18))))
{ \$lt: valor }	campo < valor	make_document(kvp("stock", make_document(kvp("\$lt", 10))))
{ \$lte: valor }	campo <= valor	make_document(kvp("precio", make_document(kvp("\$lte", 100.0))))
{ \$and: [...] }	f1 AND f2	Ver ejemplo abajo
{ \$or: [...] }	f1 OR f2	Ver ejemplo abajo
{ \$exists: true }	IS NOT NULL	make_document(kvp("email", make_document(kvp("\$exists", true))))
{ \$regex: patron }	LIKE '%x%'	make_document(kvp("nombre", make_document(kvp("\$regex", "^A"))))

```
// AND: precio > 20 Y precio < 100
auto filtro_and = make_document(
    kvp("$and", [&]() {
        auto arr = bsoncxx::builder::basic::array{};
        arr.append(make_document(kvp("precio", make_document(kvp("$gt", 20.0)))));
        arr.append(make_document(kvp("precio", make_document(kvp("$lt", 100.0)))));
        return arr.extract();
    })
);

// OR: rol = CLIENTE o rol = ADMIN
auto filtro_or = make_document(
    kvp("$or", [&]() {
        auto arr = bsoncxx::builder::basic::array{};
        arr.append(make_document(kvp("rol", "CLIENTE")));
        arr.append(make_document(kvp("rol", "ADMIN")));
        return arr.extract();
    })
);
```

6. Contar documentos

```
// Contar todos los documentos
int64_t total = col.count_documents({});
// retorna: int64_t

// Contar con filtro
auto filtro = make_document(kvp("rol", "ADMIN"));
int64_t admins = col.count_documents(filtro.view());
// retorna: int64_t

// Verificar si existe un documento
bool existe = col.count_documents(
    make_document(kvp("correo", "carlos@email.com")).view()
) > 0;
// retorna: bool

cout << "Total: " << total << endl;
cout << "Existe: " << (existe ? "Si" : "No") << endl;
```

7. Ejemplo DAO Completo en C++

```
// UsuarioDAO.hpp
#pragma once
#include <mongocxx/collection.hpp>
#include <bsoncxx/builder/basic/document.hpp>
#include <vector>
#include <string>
#include "Conexion.hpp"

using bsoncxx::builder::basic::make_document;
using bsoncxx::builder::basic::kvp;

struct Usuario {
    string nombre, correo, rol;
    int cedula;
};

class UsuarioDAO {
private:
    mongocxx::collection col;

public:
    UsuarioDAO() {
        col = Conexion::getDatabase()["usuarios"];
    }

    // INSERTAR
    bool guardar(const Usuario& u) {
        auto doc = make_document(
            kvp("nombre", u.nombre),
            kvp("correo", u.correo),
            kvp("cedula", u.cedula),
            kvp("rol", u.rol),
            kvp("activo", true)
        );
        auto r = col.insert_one(doc.view());
        return r ? true : false;
    }

    // BUSCAR POR CORREO
    bool buscarPorCorreo(const string& correo, Usuario& out) {
        auto filtro = make_document(kvp("correo", correo));
        auto r = col.find_one(filtro.view());
        if (!r) return false;
        auto doc = r->view();
        out.nombre = string(doc["nombre"].get_string().value);
        out.correo = string(doc["correo"].get_string().value);
        out.cedula = doc["cedula"].get_int32().value;
        out.rol = string(doc["rol"].get_string().value);
        return true;
    }

    // LISTAR TODOS
    vector<Usuario> listarTodos() {
        vector<Usuario> lista;
        auto cursor = col.find({});
        for (const auto& doc : cursor) {
            Usuario u;
            u.nombre = string(doc["nombre"].get_string().value);
            u.correo = string(doc["correo"].get_string().value);
            u.cedula = doc["cedula"].get_int32().value;
        }
    }
};
```



```

        lista.push_back(u);
    }
    return lista;
}

// ACTUALIZAR
bool actualizar(const string& correo, const string& nuevoNombre) {
    auto filtro = make_document(kvp("correo", correo));
    auto update = make_document(
        kvp("$set", make_document(kvp("nombre", nuevoNombre)))
    );
    auto r = col.update_one(filtro.view(), update.view());
    return r && r->modified_count() > 0;
}

// ELIMINAR
bool eliminar(const string& correo) {
    auto filtro = make_document(kvp("correo", correo));
    auto r = col.delete_one(filtro.view());
    return r && r->deleted_count() > 0;
}

// VERIFICAR SI EXISTE
bool existe(const string& correo) {
    auto filtro = make_document(kvp("correo", correo));
    return col.count_documents(filtro.view()) > 0;
}
};

```

8. Resumen de métodos y sus retornos

Método	Retorna	Nota clave
insert_one(doc)	optional<result::insert_one>	.inserted_id() → el _id generado
insert_many(docs)	optional<result::insert_many>	.inserted_count() → cantidad insertada
find(filtro)	mongocxx::cursor	Iterador — usar range-based for
find_one(filtro)	optional<document::value>	Siempre verificar con if(resultado)
update_one(f, update)	optional<result::update>	.modified_count() → 0 o 1
update_many(f, update)	optional<result::update>	.modified_count() → cantidad
replace_one(f, doc)	optional<result::replace_one>	Reemplaza el documento completo
delete_one(filtro)	optional<result::delete_result>	.deleted_count() → 0 o 1
delete_many(filtro)	optional<result::delete_result>	.deleted_count() → cantidad
count_documents(filtro)	int64_t	Contar sin traer documentos
bsoncxx::to_json(doc)	std::string	Convertir documento a JSON legible

9. Errores comunes y soluciones

Error	Causa	Solución
mongocxx::instance creado dos veces	Crear instance más de una vez	Crear una sola vez en main() o Singleton
Segfault al leer campo	Campo no existe en el documento	Verificar doc["campo"].type() antes de leer
No compila (headers no encontrados)	Rutas de include incorrectas	Usar pkg-config --cflags libmongocxx
Connection refused	MongoDB no está corriendo	sudo docker start mongod
optional vacío (nullopt)	No encontró el documento	Siempre usar if(resultado) antes de ->
Error al iterar cursor dos veces	cursor no es reutilizable	Crear nuevo find() para cada iteración